

LA COULEUR

RP01 - P26

Naim MOAWAD

Amine GACI

Rostand TOKA NZWEMFUT

Table des matières

01

Introduction

02

Les modèles

03

Conversions

04

Harmonie des couleurs

05

Code Python

06

Best Practice

07

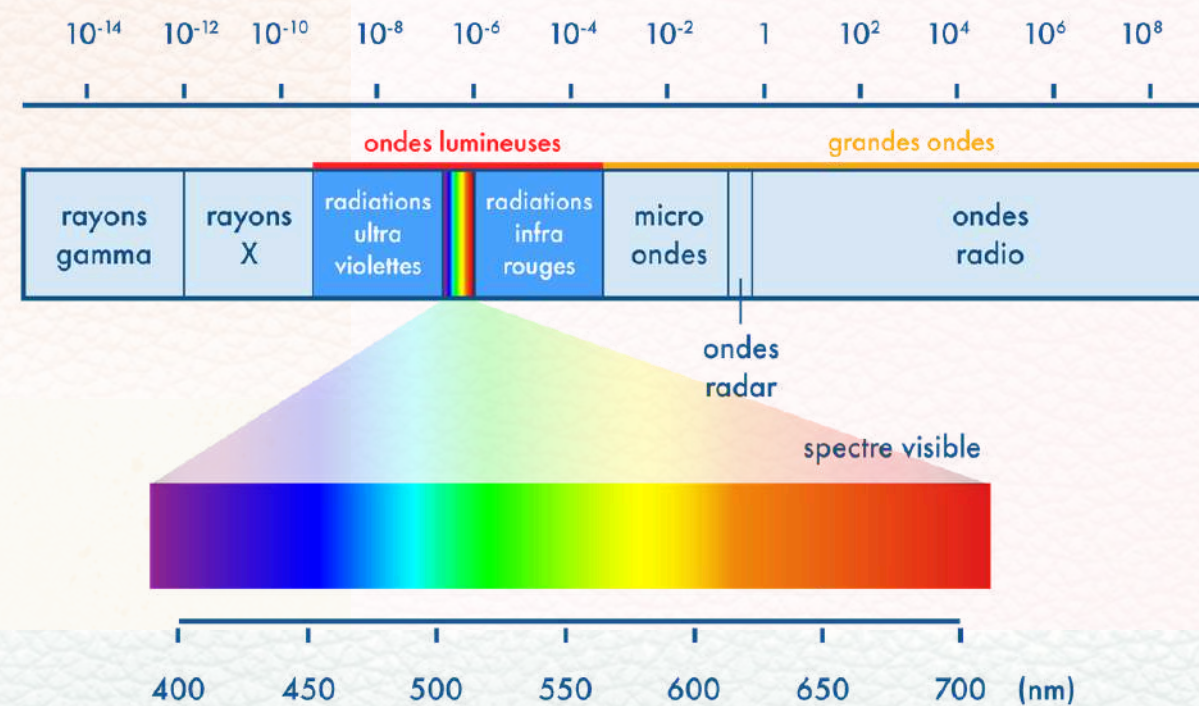
**Limites +
Améliorations**

08

Conclusion

Introduction

Spectre visible 380 nm → 750 nm



La lumière

Source électromagnétique dans la plage 380–750 nm

La matière

Absorbe ou réfléchit certaines longueurs d'onde

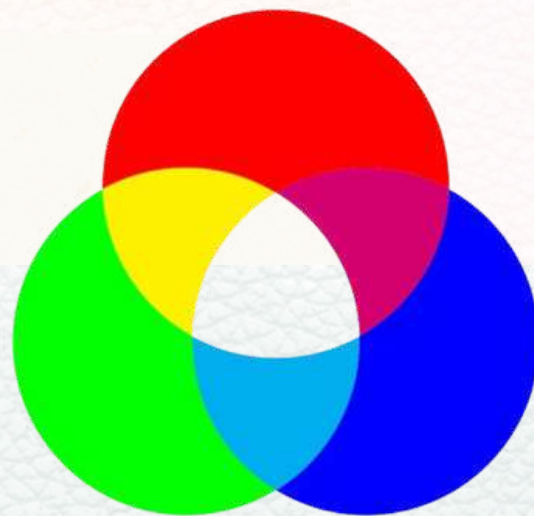
L'observateur

La rétine contient trois types de cônes : L, M, S
Sensibles respectivement au rouge, vert et bleu

Les modèles

RGB

Modèle additif : Rouge, Vert, Bleu (0-255).
Synthèse lumineuse des écrans.



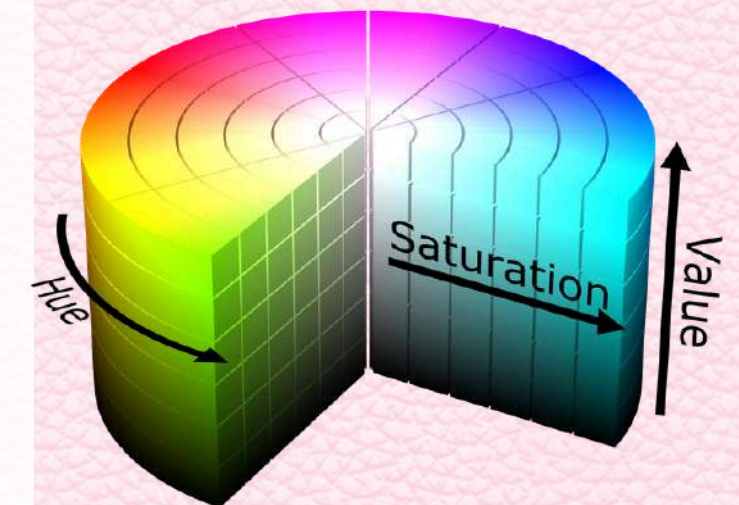
HEX

Écriture RGB en base 16 :
#RRGGBB.



HSV

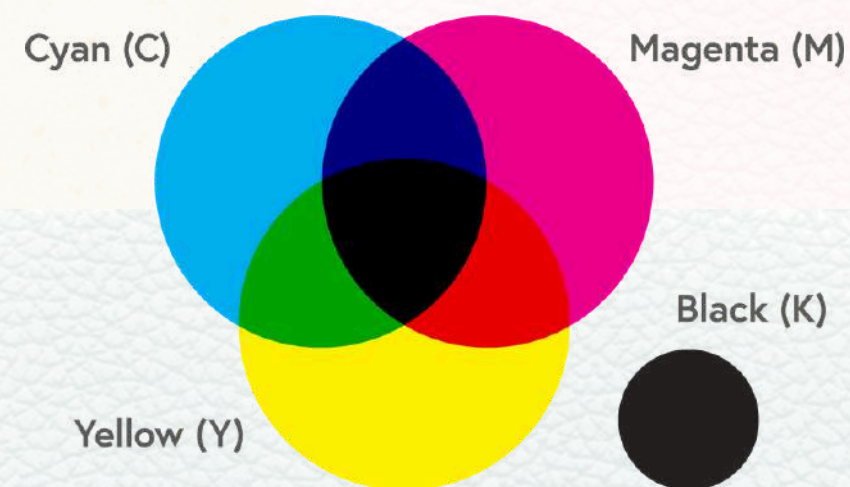
Teinte (0-360°),
saturation, valeur .
Intuitif pour le design.



Les modèles

CMYK

Modèle soustractif : Cyan, Magenta, Jaune, Noir.
Les pigments absorbent la lumière. Le noir (K) est ajouté pour renforcer les ombres et améliorer le contraste.

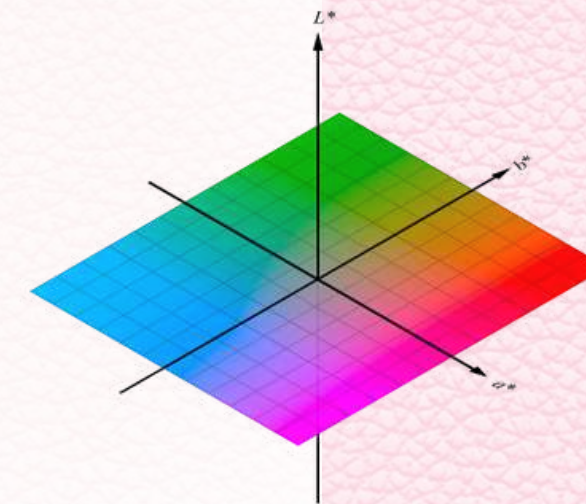


CIE Lab Fidélité à la Vision Humaine

L^* — Luminosité : de 0 (noir) à 100 (blanc).

a^* — Axe vert $\leftrightarrow$ rouge : valeurs négatives = vert, positives = rouge.

b^* — Axe bleu $\leftrightarrow$ jaune : valeurs négatives = bleu, positives = jaune.



Conversions

RGB → HSV

1. Normalisation

$$\begin{aligned}R' &= R/255 \\G' &= G/255 \\B' &= B/255\end{aligned}$$

2. Identification des valeurs maximales et minimales

$$\begin{aligned}C_{max} &= \max(R', G', B') \\C_{min} &= \min(R', G', B') \\ \Delta &= C_{max} - C_{min}\end{aligned}$$

3. Calcul de la Teinte (H)

$$H = \begin{cases} 0 & \text{si } \Delta = 0 \\ 60^\circ \times \left(\frac{G' - B'}{\Delta} \right) & \text{si } C_{max} = R' \\ 60^\circ \times \left(\frac{B' - R'}{\Delta} + 2 \right) & \text{si } C_{max} = G' \\ 60^\circ \times \left(\frac{R' - G'}{\Delta} + 4 \right) & \text{si } C_{max} = B' \end{cases}$$

4. Calcul de la Saturation (S)

$$S = \begin{cases} 0 & \text{si } C_{max} = 0 \\ \frac{\Delta}{C_{max}} & \text{sinon} \end{cases}$$

5. Calcul de la Valeur (V)

$$V = C_{max}$$

Conversions

RGB → CMYK

1. Normalisation
2. Calcul du pigment noir
3. Calcul de C,M,Y

$$R' = R/255$$

$$G' = G/255$$

$$B' = B/255$$

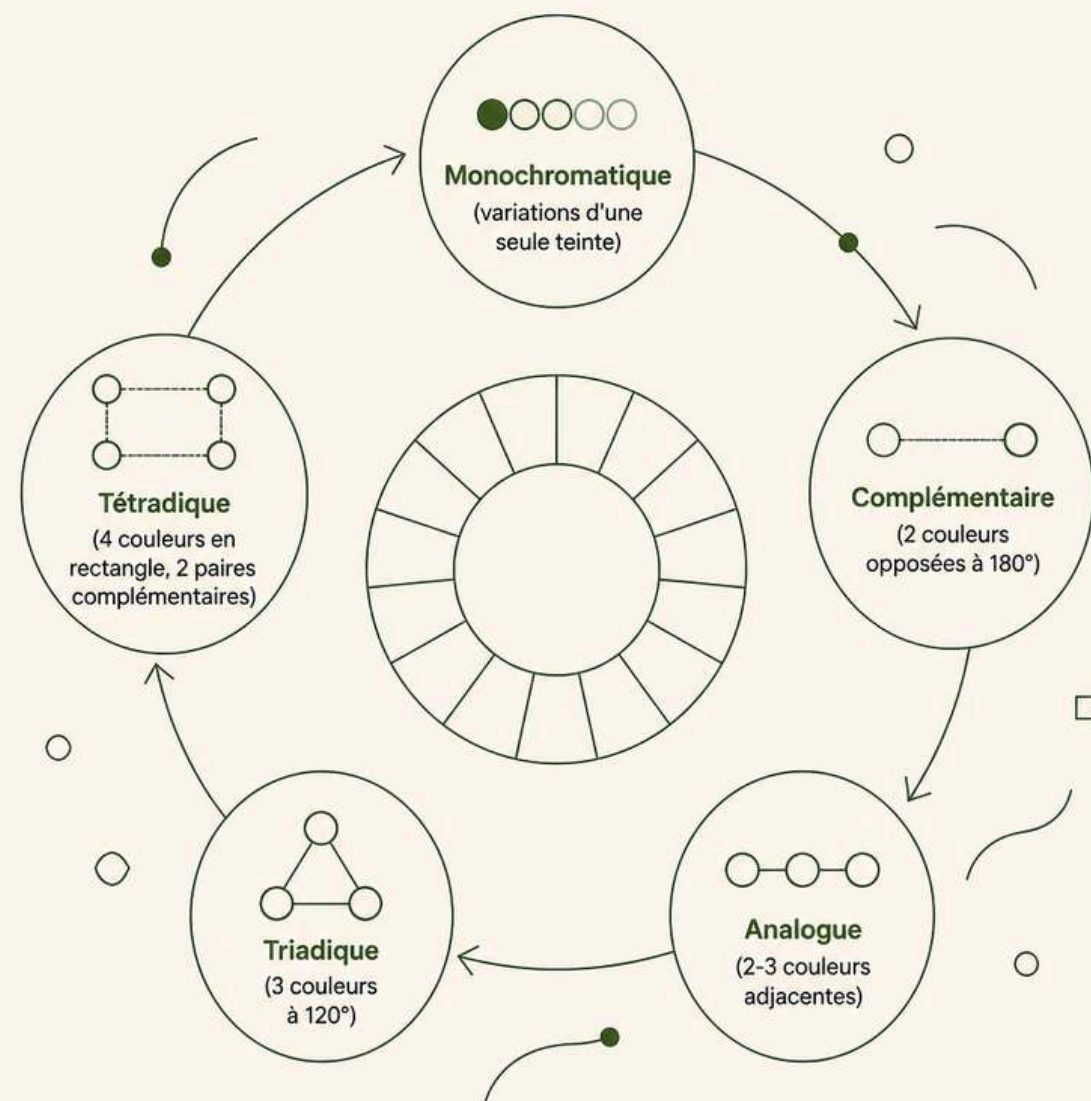
$$K = 1 - \max(R', G', B')$$

$$C = \frac{1 - R' - K}{1 - K} \quad (\text{si } K \neq 1, \text{ sinon } 0)$$

$$M = \frac{1 - G' - K}{1 - K} \quad (\text{si } K \neq 1, \text{ sinon } 0)$$

$$Y = \frac{1 - B' - K}{1 - K} \quad (\text{si } K \neq 1, \text{ sinon } 0)$$

Harmonie de couleur



Règles de sélection

- **Complémentaire** : contraste maximal (180°)
- **Analogue** : cohérence douce ($\pm 30^\circ$)
- **Triadique** : équilibre dynamique (120°)
- **Monochromatique** : variations S et L d'une teinte

Codes Python

RGB → HEX

```
def rgb_to_hex(r, g, b):  
    r = max(0, min(255, int(r)))  
    g = max(0, min(255, int(g)))  
    b = max(0, min(255, int(b)))  
  
    return f"#{r:02x}{g:02x}{b:02x}".upper()
```

RGB → CMYK

```
def rgb_to_cmyk(r, g, b):  
    r, g, b = r/255, g/255, b/255  
    k = 1 - max(r, g, b)  
    if k == 1:  
        return 0, 0, 0, 1  
    c = (1 - r - k) / (1 - k)  
    m = (1 - g - k) / (1 - k)  
    y = (1 - b - k) / (1 - k)  
    return c, m, y, k
```

RGB → HSV

```
def rgb_to_hsv(r, g, b):  
    r_prime, g_prime, b_prime = r / 255.0,  
    g / 255.0, b / 255.0  
  
    c_max = max(r_prime, g_prime, b_prime)  
    c_min = min(r_prime, g_prime, b_prime)  
    delta = c_max - c_min  
  
    h = 0  
    if delta == 0:  
        h = 0 # Gris  
    elif c_max == r_prime:  
        h = 60 * (((g_prime - b_prime) /  
delta) % 6)  
    elif c_max == g_prime:  
        h = 60 * (((b_prime - r_prime) /  
delta) + 2)  
    elif c_max == b_prime:  
        h = 60 * (((r_prime - g_prime) /  
delta) + 4)  
  
    s = 0 if c_max == 0 else delta / c_max  
    v = c_max  
  
    return h, s, v
```

RYB → RGB

Une conversion spécifique des teintes est nécessaire via une table d'interpolation.

```
RYB_TO_RGB_TABLE = [
    (0, 0), (15, 8),
    ..., (360, 360)
]

def
ryb_hsv_to_rgb(h_ryb_deg, s, v):
    h_rgb_deg =
interpolate(h_ryb_deg,
RYB_TO_RGB_TABLE)
    return
hsv_deg_to_rgb(h_rgb_deg, s, v)
```

Cercle chromatique

```
def
build_ryb_wheel(size=500, value=1.0):
    # ... calcul des
    angles (teinte RYB) et
    rayons (saturation)
    ...
    for i in
range(size):
        for j in
range(size):
            if
inside[i, j]:
                rgb =
ryb_hsv_to_rgb(h_ryb[i, j], s[i, j], value)
                img[i, j] = np.array(rgb) /
255.0
    return img
```

Harmonie complémentaire

```
def get_complementary(h, s, l):  
    # h est en degrés (0-360)  
    complementary_h = (h + 180) % 360  
    return complementary_h, s, l
```

Harmonie triadique

```
def get_triadic(h, s, l):  
    h1 = (h + 120) % 360  
    h2 = (h + 240) % 360  
    return [(h, s, l), (h1, s, l), (h2, s, l)]
```

Harmonie analogique

```
def get_analogous(h, s, l, angle=30):  
    # angle en degrés (ex: 30)  
    h1 = (h - angle + 360) % 360  
    h2 = (h + angle) % 360  
    return [(h1, s, l), (h, s, l), (h2, s, l)]
```

Harmonie monochromatique

```
def get_monochromatic(h, s, l, steps=3):  
    # Prend une couleur HSL et retourne  
    'steps' variations  
    # Ici, nous varions la luminosité (L)  
    et la saturation (S)  
    palette = []  
    for i in range(steps):  
        # Diminuer L et augmenter S pour  
        des variations plus foncées/saturées  
        # ou ajuster selon le besoin. Ex:  
        varier S de 10% et L de 15%  
        new_s = max(0, min(1, s - 0.1 *  
            (steps - 1 - i)))  
        new_l = max(0, min(1, l - 0.15 *  
            (steps - 1 - i)))  
        palette.append((h, new_s, new_l))  
    return palette
```

```
# Exemple d'utilisation (retourne HSL)  
# get_monochromatic(240, 0.8, 0.5) # Bleu  
vibrant avec des variations
```

Best Practice

Couleur de base

Color picker natif + saisie HEX + 6
préréglages (Marine, Forêt,
Bordeaux, Charbon, Indigo, Cuivre)

4 harmonies auto

Complémentaire, Analogue,
Triadique, Monochrome - 3
couleurs chacune, mises à jour en
temps réel.

Sliders S & L

Ajustement saturation et
luminosité globaux. Bouton Reset
pour revenir aux valeurs neutres.

Fond du logo

6 préréglages (Blanc, Crème,
Ardoise, Marine, Noir...) + color
picker libre pour tester tous les
contextes.

Sliders S & L

Ajustement saturation et
luminosité globaux. Bouton Reset
pour revenir aux valeurs neutres.

Best Practice

Limites

- L'œil humain ne perçoit qu'une partie du spectre lumineux (380–750 nm).
- Les écrans (RGB) et les imprimantes (CMYK) ne reproduisent pas toutes les couleurs réelles.
- Certaines couleurs RGB sont impossibles à imprimer en CMYK.
- Les conversions entre espaces colorimétriques entraînent des pertes de précision.
- Le modèle HSV n'est pas perceptuellement uniforme.
- Le rendu dépend du matériel utilisé (écran, imprimante, éclairage).

Exemple

RGB (100,150,200) → CMYK(99,151,199)

La couleur obtenue n'est pas exactement identique à la couleur d'origine.

Améliorations

- Utiliser des espaces colorimétriques adaptés (Lab, XYZ).
- Employer des profils ICC pour la gestion des couleurs.
- Calibrer régulièrement les écrans et imprimantes.
- Vérifier les contrastes selon les normes WCAG.
- Utiliser des harmonies de couleurs basées sur le cercle chromatique.
- Tester les couleurs sur plusieurs supports avant validation.
- Automatiser la génération de palettes harmonieuses grâce à Python.
- **Perspective**
- **Développement d'outils intelligents capables de proposer automatiquement des palettes optimisées pour le web, l'impression et l'accessibilité.**

Conclusion

**Merci pour
votre attention**